

Кейс 2 (Неделя 2). Индивидуальное задание

REST API на Express: сервис учёта заявок на обслуживание оборудования

Формат выполнения: индивидуально. Ориентировочное время — 6 часов. Результат сдаётся в виде репозитория с историей веток и Pull Request.

Проверяемые темы недели: Express (сервер, роутинг, слоистая структура проекта); REST-принципы и CRUD-эндпоинты (GET/POST/PUT/PATCH/DELETE); тестирование эндпоинтов в Postman; политики безопасности — CORS, rate limiting, SameSite, Secure Context; middleware, валидация входных данных, обработка ошибок, логирование.

Связь с другими кейсами: кейс развивает результат Кейса 1 — модуль работы с внешним погодным API переиспользуется как сервис внутри приложения. На Неделе 3 хранилище этого же API переносится на PostgreSQL через Sequelize, поэтому слой доступа к данным сразу изолируется за репозиторием.

1. Условие

Требуется разработать REST API на Express для учёта заявок на техническое обслуживание оборудования производственной площадки (например, ветропарка). Сервис ведёт справочник оборудования и заявки на его обслуживание, контролирует жизненный цикл заявки и позволяет оценить погодные условия на объекте перед планированием наружных работ.

Данные на этой неделе хранятся в JSON-файле или в памяти процесса — база данных подключается на Неделе 3. Доступ к данным выполняется только через слой репозитория, чтобы последующая замена хранилища не затрагивала остальной код.

2. Модель данных

Оборудование (equipment)

id	string (uuid, генерируется сервером)
name	string, 3-100 символов, обязательное
type	turbine inverter sensor substation
serialNumber	string, уникальный в пределах системы
location	{ lat: number, lon: number }
status	operational maintenance fault decommissioned
installedAt	ISO-дата, не в будущем

Заявка на обслуживание (maintenance request)

id	string (uuid, генерируется сервером)
equipmentId	string, ссылка на существующее оборудование
title	string, 5-120 символов, обязательное
description	string, до 2000 символов
priority	low medium high critical
status	new in_progress done rejected (по умолчанию new)
plannedAt	ISO-дата-время, необязательное
createdAt	ISO-дата-время (проставляется сервером)
updatedAt	ISO-дата-время (проставляется сервером)

Допустимые переходы статуса заявки:

new → in_progress → done ; new → rejected ; in_progress → rejected

Из статусов `done` и `rejected` переходы запрещены. Попытка недопустимого перехода — ответ 409.

3. Состав API

Метод	Путь	Назначение
GET	/api/health	Проверка доступности сервиса
GET	/api/equipment	Список оборудования: фильтры, сортировка, пагинация
POST	/api/equipment	Создание единицы оборудования
GET	/api/equipment/:id	Карточка оборудования
PATCH	/api/equipment/:id	Частичное обновление
DELETE	/api/equipment/:id	Удаление (запрещено при наличии открытых заявок)
GET	/api/equipment/:id/request s	Заявки по конкретной единице оборудования
GET	/api/equipment/:id/weather	Прогноз по координатам объекта и пригодность окна для наружных работ
GET	/api/requests	Список заявок: фильтры, сортировка, пагинация
POST	/api/requests	Создание заявки
GET	/api/requests/:id	Карточка заявки
PATCH	/api/requests/:id	Редактирование полей заявки
PATCH	/api/requests/:id/status	Смена статуса заявки с проверкой допустимости перехода
DELETE	/api/requests/:id	Удаление заявки

Требования к эндпоинту прогноза: сервис берёт координаты оборудования, обращается к внешнему погодному API (модуль из Кейса 1) и возвращает прогноз вместе с признаком пригодности окна для наружных работ. Правило пригодности задаётся в конфигурации (например, отсутствие осадков и скорость ветра ниже порога) и описывается в README. Ошибка или недоступность внешнего API не должна приводить к падению сервиса: возвращается корректный код ответа и понятное сообщение.

4. Функциональные требования

1. Списочные эндпоинты поддерживают фильтрацию (по статусу, типу, приоритету, идентификатору оборудования, диапазону дат), сортировку и пагинацию. Ответ содержит данные и метаданные: `total`, `page`, `limit`.
2. Создание заявки на несуществующее оборудование отклоняется с кодом 404, создание оборудования с уже занятым серийным номером — с кодом 409.
3. Удаление оборудования, по которому есть незакрытые заявки, запрещено (409).
4. Изменение статуса выполняется отдельным эндпоинтом и проверяется по таблице допустимых переходов.
5. Поля `id`, `createdAt` и `updatedAt` проставляются сервером и не могут быть изменены через API. Неизвестные поля тела запроса игнорируются.

6. Все ответы об ошибках имеют единый формат, например:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Некорректные данные запроса",
    "details": [ { "field": "priority", "message": "Недопустимое значение" } ],
    "requestId": "b1f2c3d4"
  }
}
```

5. Технические требования

Архитектура

- Слоистая структура: маршруты → контроллеры → сервисы → репозитории. Бизнес-логика в сервисах, работа с данными — только в репозиториях.
- Пример структуры: `src/app.js`, `src/server.js`, `src/routes/`, `src/controllers/`, `src/services/`, `src/repositories/`, `src/middlewares/`, `src/validators/`, `src/errors/`, `src/config/`.
- Сборка приложения отделена от запуска сервера: модуль `app` экспортируется и может быть подключён в тестах.
- Конфигурация — только через переменные окружения (`PORT`, `NODE_ENV`, `CORS_ORIGINS`, `RATE_LIMIT_*`, `WEATHER_API_URL`, `REQUEST_TIMEOUT_MS`). В репозитории — `.env.example`.

Middleware и обработка ошибок

- Логирование запросов, разбор JSON с ограничением размера, присвоение идентификатора запросу, валидация, обработчик 404, централизованный обработчик ошибок — в указанном порядке.
- Валидация выполняется отдельным переиспользуемым middleware (собственная реализация или библиотека уровня `Joi` / `Zod` / `express-validator`) и покрывает `body`, `params` и `query`.
- Ошибки приложения описаны собственными типами (`NotFoundError`, `ValidationError`, `ConflictError`) и преобразуются в HTTP-ответ только в обработчике ошибок.
- Асинхронные ошибки контроллеров передаются в обработчик ошибок, необработанных отклонений промисов нет.
- В режиме `production` в ответ не попадают стек-трейсы и внутренние сообщения.

Безопасность

- CORS: явный список разрешённых источников из переменной окружения, а не «*». В README поясняется, какие источники и почему разрешены.
- Ограничение частоты запросов на маршруты `/api` с ответом 429 и заголовками лимита.
- Ограничение размера тела запроса и защитные HTTP-заголовки (`helmet` или ручная настройка).
- Если в решении используются cookie — флаги `HttpOnly`, `Secure`, `SameSite` выставлены осознанно, выбор значения `SameSite` поясняется.

Логирование

- Каждый запрос логируется: метод, путь, код ответа, длительность, идентификатор запроса.

- Ошибки логируются с идентификатором запроса, который возвращается клиенту и позволяет найти запись в логах.
- Используются уровни логирования; отладочные вызовы `console.log` в итоговом коде отсутствуют.

6. Тестирование в Postman

- Коллекция покрывает все эндпоинты и сгруппирована по ресурсам.
- Используются переменные окружения: `{{baseUrl}}`, идентификаторы созданных сущностей передаются между запросами.
- Сохранены негативные сценарии: некорректное тело (400/422), несуществующий идентификатор (404), недопустимый переход статуса и дубль серийного номера (409), превышение лимита частоты (429).
- В запросах написаны автотесты `pm.test` — минимум на код ответа и структуру тела.
- Коллекция экспортирована в JSON и добавлена в репозиторий (`docs/postman/`).

7. Требования к оформлению в Git

- Работа разбита минимум на четыре ветки по функциональным блокам (например: `feat/equipment-crud`, `feat/requests-crud`, `feat/validation-errors`, `feat/security-logging`).
- Каждый блок вливается в `main` отдельным Pull Request с описанием: что сделано, как проверить.
- Все PR проходят `code review`; замечания исправлены или аргументированно закрыты в комментариях.
- Коммиты атомарные, сообщения в едином стиле; прямых коммитов в `main` нет.

8. Требования к README

README.md должен содержать: описание сервиса, требования к окружению, установку и запуск, перечень переменных окружения, таблицу эндпоинтов, модель данных и схему переходов статусов, формат ответа об ошибке, примеры запросов и ответов (в том числе ошибочных), описание правил безопасности (CORS, лимиты) и структуры проекта.

9. Дополнительная часть (по желанию)

- Запуск приложения через Docker и Docker Compose.
- Автотесты API на Jest + Supertest для основных сценариев.
- Аутентификация по API-ключу или токenu для изменяющих операций (POST, PATCH, DELETE).
- Эндпоинт массового импорта заявок с частичным успехом: отчёт по каждой записи.
- Простая HTML-страница, работающая с API через `fetch`: список заявок с фильтрами и форма создания.

10. Что сдаётся

1. Ссылка на репозиторий с README и `.env.example`.
2. Ссылки на влитые Pull Request с пройденным `code review`.
3. Экспортированная коллекция Postman в составе репозитория.

4. Устная защита 10–15 минут: демонстрация сценариев через Postman, разбор архитектуры и ответы на вопросы по коду.

11. Критерии оценивания

Каждый критерий оценивается по шкале от 0 до 3 баллов:

Балл	Значение
0	Критерий не выполнен
1	Критерий выполнен частично
2	Критерий выполнен полностью
3	Критерий перевыполнен (решение лучше требуемого)

Итоговый результат = сумма по всем критериям (балл × вес). Максимум по обязательным критериям — 145,5 балла (48,5 × 3). Бонусные критерии добавляют до 10,5 баллов сверх этого.

Критерий	Вес
Архитектура и структура проекта	
Реализована слоистая архитектура (маршруты → контроллеры → сервисы → репозитории); бизнес-логика не находится в обработчиках маршрутов	2
Слой доступа к данным изолирован за интерфейсом репозитория: замена источника данных не затрагивает сервисы и контроллеры	1,5
Сборка приложения (app) отделена от запуска сервера (server), что позволяет подключать приложение в тестах	0,5
Конфигурация вынесена в переменные окружения (порт, режим, CORS-origin, лимиты), в репозитории есть .env.example, хардкода нет	1
Модуль обращения к внешнему погодному API из Кейса 1 переиспользован как отдельный сервис, а не скопирован в контроллер	1
REST-дизайн и CRUD	
URL и HTTP-методы соответствуют REST: ресурсы во множественном числе, идентификатор в пути, глаголы в URL отсутствуют	2
Полный CRUD реализован для обеих сущностей (список, чтение, создание, обновление, удаление)	2
Коды ответов используются корректно: 200, 201 с заголовком Location, 204, 400, 404, 409, 422	2
Список поддерживает фильтрацию, сортировку и пагинацию; параметры запроса валидируются, ответ содержит метаданные (total, page, limit)	1,5
Реализован вложенный ресурс: заявки по конкретной единице оборудования	0,5
Переходы статусов заявки контролируются на уровне сервиса, недопустимый переход отклоняется с кодом 409	1,5
Ответы имеют единообразную структуру (одинаковая форма объекта и списка во всех эндпоинтах)	1

Критерий	Вес
Middleware, валидация и обработка ошибок	
Валидация входных данных вынесена в переиспользуемый middleware, проверяются body, params и query	2
Ошибка валидации возвращает перечень некорректных полей с причинами, а не общее сообщение	1
Неизвестные поля тела запроса отбрасываются, изменение служебных полей (id, даты создания) через API невозможно	1
Реализован централизованный обработчик ошибок; формат ответа об ошибке единый для всего API	2
Ошибки приложения описаны отдельными типами (не найдено, конфликт, ошибка валидации) и не привязаны к Express напрямую	1
Асинхронные ошибки контроллеров не теряются (обёртка над обработчиками или явный перехват), необработанных исключений нет	2
Подключён обработчик несуществующих маршрутов, возвращающий 404 в том же формате ошибки	0,5
Порядок подключения middleware осмыслен и обоснован участником	0,5
Политики безопасности	
CORS настроен с явным списком разрешённых источников и методов, значение задаётся через переменную окружения	1
Подключено ограничение частоты запросов: при превышении возвращается 429 с соответствующими заголовками	1
Ограничен размер тела запроса, установлены защитные HTTP-заголовки (helmet или эквивалент)	1
Если используются cookie — установлены флаги HttpOnly, Secure и SameSite с пояснением выбора значения	0,5
Секреты и параметры окружения отсутствуют в репозитории, во внешнем ответе нет стек-трейсов и внутренних деталей	1
Логирование	
Каждый входящий запрос логируется: метод, путь, код ответа, длительность	1
Каждому запросу присваивается идентификатор, который возвращается в ответе об ошибке и позволяет найти запрос в логах	1
Используются уровни логирования, отладочный вывод в консоль отсутствует	0,5
Тестирование в Postman	
Коллекция покрывает все эндпоинты, сгруппирована по ресурсам, использует переменные окружения ({{baseUrl}}, {{id}})	1,5
Сохранены проверки негативных сценариев: 400, 404, 409, 422, 429	1,5
В запросах есть автотесты (pm.test) хотя бы на код ответа и структуру тела, идентификаторы передаются между запросами через переменные	1
Коллекция экспортирована и добавлена в репозиторий	0,5
Git	

Критерий	Вес
История коммитов атомарная, сообщения осмысленные и в едином стиле	1
Работа велась в отдельных ветках по задачам, прямых коммитов в main нет	1
Оформлены Pull Request с описанием изменений и способа проверки	1,5
Документация	
README содержит установку, переменные окружения, запуск, таблицу эндпоинтов и примеры запросов и ответов	1,5
Описаны модель данных, допустимые статусы и правила переходов, а также формат ошибок	1
Качество кода и защита решения	
Код читаемый: единый стиль, осмысленные имена, отсутствие дублирования и мёртвого кода	1
Участник объясняет архитектуру и принцип работы любой части приложения, обосновывает выбор кодов ответов и структуры слоёв	2
Дополнительные задания (бонус)	
Проект и его окружение запускаются через Docker и Docker Compose	1
Написаны автотесты API (Jest + Supertest) на основные сценарии	1
Добавлена аутентификация по API-ключу или токену для изменяющих операций	0,5
Реализован эндпоинт массового импорта заявок с частичным успехом (207 / отчёт по каждой записи)	0,5
Сделана простая HTML-страница, работающая с API через fetch (список заявок и создание новой)	0,5
Сумма весов: обязательные критерии	48,5
Сумма весов: бонусные критерии	3,5

12. Шкала итоговой оценки

Взвешенная сумма баллов	Итоговая оценка
124,0 и выше (85 % и более)	Отлично
от 102,0 до 123,5 (70–84 %)	Хорошо
от 73,0 до 101,5 (50–69 %)	Удовлетворительно
менее 73,0 (менее 50 %)	Не зачтено, требуется доработка

Процент считается от максимума по обязательным критериям. Бонусные баллы прибавляются к итоговой сумме и могут повысить оценку.